

SORTING ALGORITHMS

Sorting $\rightarrow$ Process of ordering / arranging the given element in a particular sequence / order as the given criteria.

[50, 79, 3005, 452, 297]

Increasing Order $\rightarrow$ 50 79 297 452 3005

Decreasing Order $\rightarrow$ 3005 452 297 79 50

$\rightarrow$ Arrange given elements in increasing order of their 1st digit.

$\rightarrow$ 297 ~~50~~ 452 50 79
3005

Sorting Algorithms:

- 1) Bubble
- 2) Selection
- 3) Insertion
- 4) Merge
- 5) Quick
- 6) Heap
- 7) Radix

Dnc...

Misc...

For every sorting Algo.

- $\rightarrow$ Example
- $\rightarrow$ Code
- $\rightarrow$ Internal working
- $\rightarrow$ Complexity analysis
 - $\hookrightarrow$ Time
 - $\hookrightarrow$ Space
- $\rightarrow$ Properties

Comparison-Based Sorting:

When an element is compared with other element of given data to achieve sorting.

Non-Comparison Based Sorting:

When sorting is achieved without any comparison among the elements to be sorted... [Radix Sort]

Other classification of Sorting Algorithms:

Internal v/s External
Comparison v/s Non Comp.
Recursive v/s Iterative
Inplace v/s not-inplace
Stable v/s unstable.

Note: If nothing is specified, default sorting order will be as ending / increasing

* Inplace Sorting Algorithm

Additional / Auxiliary space Requirement

- At most $O(1)$ (excluding recursion track)
- Recursion stack of at most $O(\log n)$

→ At most $O(1)$ [Space complexity]
↳ excluding Recu.
 $O(\log n)$
↳ including Recu.

Merge Sort := Space complexity $\Rightarrow O(n + \log n)$

↳ Not Inplace

* Stable Sorting Algorithm

If relative position of two equal elements is maintained before and after sorting

• Formal Definition

If $A[i] = A[j]$ & In given input: $A[i]$ is before $A[j]$
Then, algorithm is stable if $A[i]$ is before $A[j]$ is finally sorted (after sorting array as well).

Example

Given I/p Array to be sorted

5	10	7	15	7	2
1	2	3	4	5	6

Before Sorting

2	5	7	7	10	15
---	---	---	---	----	----

Stable Sorting

After Sorting

2	5	7	7	10	15
---	---	---	---	----	----

Unstable Sorting

* Inversion in an Array

Formal Definition:

- A pair of indices (i, j) is considered to be an inversion of the given array $A[1, \dots, n]$ if $i < j$ and $A[i] > A[j]$

Note: i & j need not be adjacent indices (not mandatory)

In simple terms, if a larger element is present before a smaller element, then their indices will form an inversion pair

Example 1 2 3 4 5
 A:

10	12	6	5	2
----	----	---	---	---

Total no of inversion pairs in given array A?

- (1,3) (1,4) (1,5) → 3
 (2,3) (2,4) (2,5) → 3
 (3,4) (3,5) → 2
 (4,5) → 1

∴ Total number of inversion
 ⇒ 3+3+2+1 = 9

If array size is $A[n]$, then what are maximum & minimum inversion.

Minimum no. of Inversion.
 → A is in increasing order

A:

5	10	15	18
---	----	----	----

Inv: $\downarrow \downarrow \downarrow \downarrow$
 0 0 0 0
 N=4
Total inversion = 0

Maximum no. of inversions
 → A is in decreasing order

A:

18	15	10	5
----	----	----	---

$\downarrow \downarrow \downarrow$
 $n-1 \quad n-2 \quad n-3$

Nelement.

In general..

$$\left(\sum_{i=1}^n i \right) - n = \frac{n(n+1)}{2} - n$$

∴ A → n

$$0 \leq |\text{No. of Inversion}| \leq \frac{n(n-1)}{2}$$

$$\begin{aligned} \Rightarrow \left(\sum_{i=1}^n i \right) - n &= \frac{n(n+1)}{2} - n \\ &= \frac{n^2 + n - 2n}{2} \\ &= \frac{n^2 - n}{2} \Rightarrow \frac{n(n-1)}{2} \end{aligned}$$

Time complexity of any comparison based sorting Algo:

→ depends on two imp factors.

- (i) No. of comparisons
- (ii) No. of swaps (No. of inversion)

	$(n-1) + (n-2) + \dots + 1$	$\frac{n(n-1)}{2}$
	$(n-1) + (n-2) + \dots + 1$	$\frac{n(n-1)}{2}$
	$(n-1) + (n-2) + \dots + 1$	$\frac{n(n-1)}{2}$
	$(n-1) + (n-2) + \dots + 1$	$\frac{n(n-1)}{2}$
	$(n-1) + (n-2) + \dots + 1$	$\frac{n(n-1)}{2}$

1. Bubble Sort

By default : Aescending Order

- Basic Algorithm... (Algo-1)
- Better Algorithm... (Algo-2)
- Optimized Bubble Sort... (Algo-3)

Logic/Idea... ⇒ In every i^{th} pass/iteration, place the i^{th} max element at its correct position....

Imp. Observation... After pass 1 → 1 element
After pass 2 → 2 elements

After pass $(n-1)^{th}$ → $(n-1)$ elements will be at their correct position

Note : The Max number of passes required to = $(n-1)$ passes.

Given: A:

1	2	3	4	5	6
75	55	15	10	35	5

Given array is input to pass 1.

After pass-1 :

Pass-2 :

Pass-3 :

Pass-4 :

Pass-5 :

75	55	15	10	35	5
55	15	10	35	5	75
15	10	35	5	55	75
10	15	5	35	55	75
10	5	15	35	55	75
5	10	15	35	55	75

(Algo:1) Basic Bubble Sort:

1 based Indexing

AlgoBubbleSort(A, n)

```

for (pass = 1; pass <= (n-1); pass++)
{
    for (j = 1; j <= (n-1); j++)
    {
        if (A[j] > A[j+1])
        {
            swap(A[j], A[j+1])
        }
    }
}
    
```

1 → n-1
1 → n-1
1 → n-1
⋮
1 → n-1

SORTED

(For Algo:1)

Algorithm Time Complexity

	No. of Comparison	No. of Swaps
Inc. Best Case.	$(n-1) * (n-1)$ $= (n-1)^2$	0
Dec. Worst Case.	$(n-1) * (n-1)$ $= (n-1)^2$	$\frac{n(n-1)}{2}$ $= (n-1) + (n-2) + \dots$

(Algo: 2): Better Algorithm... of Bubble Sort.

Algo BubbleSort (A, n)

for (pass = 1; pass <= (n-1); pass++)

for (j = 1; j <= (n-pass); j++)

if (A[j] > A[j+1])

swap (A[j], A[j+1])

}

1 → n-1

1 → n-2

1 → n-3

⋮

⇒ We have to reduce the unnecessary comparison in Best case.

Issue with Algo1 and Algo2:

A: [10 | 20 | 25 | 40]

Total Cmp: 3

Total Swap: 0

(Algo: 3): Optimized Bubble Sort.

Algo BubbleSort (A, n)

for (pass = 1; pass <= (n-1); pass++)

flag = 0

for (j = 1; j <= (n-pass); j++)

if (A[j] > A[j+1])

swap (A[j], A[j+1])

flag = 1

// Inversion was present

if (flag == 0) break;

// There is not any inversion

For Algo (2):

Time complexity Analysis...

	No. of Comparisons	No. of Swaps (Inversion)
Best Case Asc. order	$\frac{n(n-1)}{2} +$	0 } $\rightarrow \Omega(n^2)$ still.
Worst Case Desc. order	$\frac{n(n-1)}{2} +$	$\frac{n(n-1)}{2}$ } $\rightarrow O(n^2)$ still.

• **Optimized Bubble Sort (Algo 3)**

If there are no swaps in any of the pass, then the array has got sorted already

For Algo (3):

Time Complexity Analysis.

Best case: 1 pass, 0 swap, (n-1) comparisons.

	No. of comparison	No. of Swaps (Inversion)
Best Case Asc. order	$(n-1) +$	0 } $\rightarrow \Omega(n)$
Worst Case Desc. order	$\frac{n(n-1)}{2} +$	$\frac{n(n-1)}{2}$ } $\rightarrow O(n^2)$

A: [100, 85, 10, 25, 45, 50]

Algo 1 & Algo 2 (5 passes)

Pass 1: [85, 10, 25, 45, 50, 100]

Pass 2: [10, 25, 45, 50, 85, 100]

Pass 3: [10, 25, 45, 50, 85, 100]

Pass 4: [10, 25, 45, 50, 85, 100]

Pass 5: [10, 25, 45, 50, 85, 100]

Algo 3 (3 passes)

Pass 1: [85, 10, 25, 45, 50, 100] (flag=1)

Pass 2: [10, 25, 45, 50, 85, 100] (flag=1)

Pass 3: [10, 25, 45, 50, 85, 100] (flag=0) break.

Properties

→ Space Complexity: $O(1)$

→ Inplace ✓

→ Stable ✓

2. Selection Sort

Steps:

- (1) In i^{th} pass, find the index / position of the i^{th} smallest element.
- (2) Swap it with the element that is present in the i^{th} index.
- (3) Keep repeating until array is sorted.

Imp.

Observations:

After 1st pass → 1st smallest will be placed at 1st pos.

After 2nd pass → 2nd " " " 2nd pos.

⋮

After $(n-1)^{\text{th}}$ pass → $(n-1)^{\text{th}}$ " " " $(n-1)^{\text{th}}$ pos

Algo Selection sort (A, n)

for (pass = 1; pass <= (n-1); pass++)

min-Ind = pass // initialize

for (j = pass+1; j <= n; j++)

if (A[j] < A[min-Ind])

min-Ind = j

// index of min element found so far

swap(A[pass], A[min-Ind]) // place passth min at the correct position

A: 60 90 15 10 30 45

Pass 1: 10 90 15 60 30 45

Pass 2: 10 15 90 60 30 45

Pass 3: 10 15 30 90 60 45

Pass 4: 10 15 30 45 90 60

Pass 5: 10 15 30 45 60 90

Sorted...

Time Complexity Analysis...

	No. of Comparisons.	No. of swaps (Inversions)
Best Case	$\{0(n^2)\}$	$\{0(n^2)\}$
Worst Case	$\{0(n^2)\}$	$\{0(n^2)\}$

Properties:

→ Space complexity: $O(1)$ Constant.

→ Inplace ✓

→ Unstable

→ Always taken $(n-1)$ passes.

→ Every Pass, exactly 1 swap

Total = $1 * (n-1) = (n-1)$ swaps to get sorted

→ Among other Algo..., taken least swaps in worst case $\rightarrow (n-1) + O(n)$

7 7 2

P1: 2 7 7

P2: 2 7 7
(unstable)

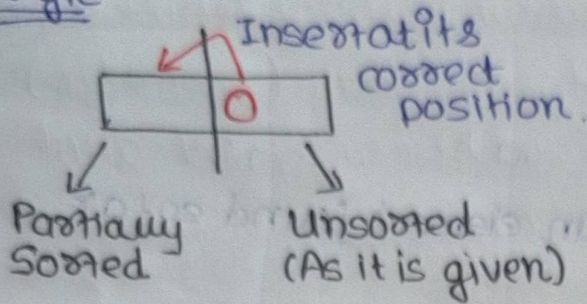
3. Insertion Sort

TC: $\frac{5n^2}{2} O(n^2)$
 BC $\rightarrow n(n)$
 WC $\rightarrow O(n^2)$

Properties:

- $\rightarrow$ Space Complexity $\Rightarrow O(1)$
- $\rightarrow$ Inplace $\checkmark$
- $\rightarrow$ Stable $\checkmark$
- $\rightarrow$ Always take $(n-1)$ passes
- $\rightarrow$ If input is pre-sorted, then it takes time of $O(n+d)$ where
 $n = \text{no. of elements}$
 $d = \text{no. of inversion}$

Logic



Insertion Sort Code: 1 Bounded Indexing...

Algo InsertionSort(A, n)

```

for (j = 2; j <= n; j++)
{
    key = A[j]
    i = j - 1
    while (i > 0 and A[i] > key)
    {
        A[i+1] = A[i]
        i--
    }
    A[i+1] = key
}
    
```

85 22 49 97 86 15

Pass 1: 85 | 22 49 97 86 15

Pass 2: 22 85 | 49 97 86 15

Pass 3: 22 49 85 | 97 86 15

Pass 4: 22 49 85 86 | 97 15

Pass 5: 22 49 85 86 97 | 15

(15, 22, 49, 85, 86, 97) Sorted

$(n-1)$ passes

	No. of Comparison	No. of Swaps (Inversion)	
Best Case	$\{(n-1) + \}$	$0\}$	$\rightarrow O(n)$
Worst Case	$\{\frac{n(n-1)}{2} + \}$	$\{\frac{n(n-1)}{2}\}$	$\rightarrow O(n^2)$

Day Run of previous Insertion Sort

I/p: A = 80 | 20 40 90 30 60

P1: j = 2;

key = A[j] = A[2] = 20

i = j - 1

= 2 - 1 = 1

while (i > 0 and A[i] > key)

A[i+1] = A[j]

i--

A[i+j] = key

// A[0+1] = 20

P2: j = 3

key = A[3] = 40

i = j - 1 = 3 - 1 = 2

A[i+1] = 40

P3: j = 4

key = A[4] = 90

i = j - 1 = 4 - 1 = 3

A[3+1] = key = 90

A[4] = 90

No swap.

4. Radix Sort. (Non-Comparison Based).

- No. of passes = no. of digits in max element of given input.
- make use of Bucket / Bins.

Example

Base = 10 (Decimal)

I/p: A = 64, 723, 99, 83, 545, 65, 333, 7, 4, 124

Input: $\{L \rightarrow R\}$

O/p: Bucket $\{L \rightarrow R, \text{Bottom to Top}\}$

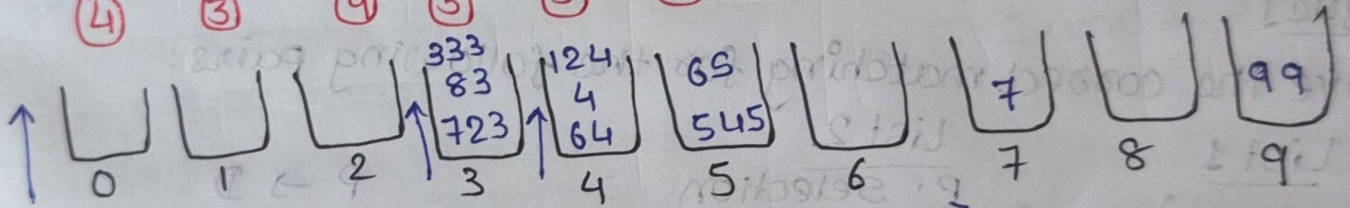
max(A) = 723

no. of digit (723) = 3 → means 3 passes

Pass 1:

Bucket as per unit place digit (Right most digit)

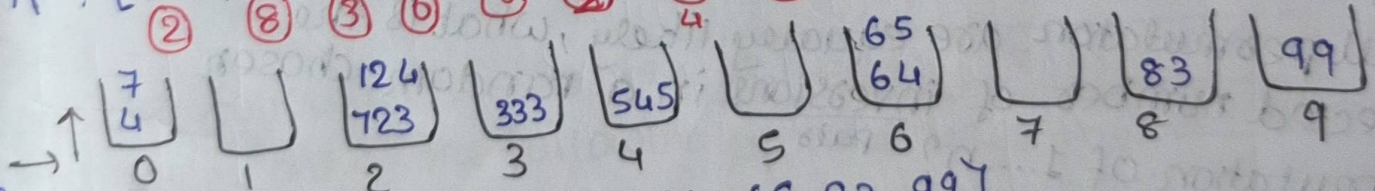
I/p: 64, 723, 99, 83, 545, 65, 333, 7, 4, 124
(4) (3) (9) (3) (5) (5) (3) (7) (4) (4)



O/p: [723, 83, 333, 64, 4, 124, 545, 65, 7, 99]

Pass 2:

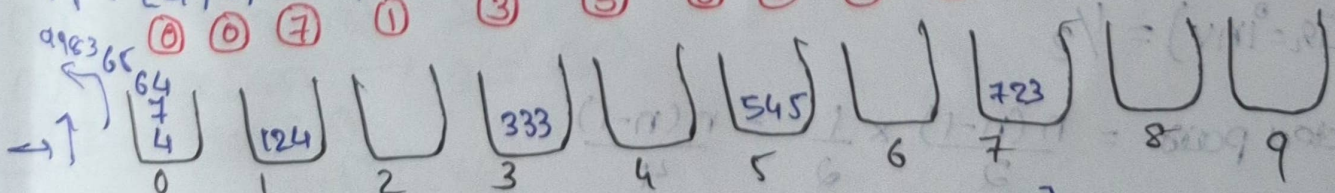
I/p: [723, 83, 333, 64, 4, 124, 545, 65, 7, 99]
(2) (8) (3) (6) (0) (2) (4) (5) (0) (9)



O/p: [4, 7, 723, 124, 333, 545, 64, 65, 83, 99]

Pass 3:

I/p: [4, 7, 723, 124, 333, 545, 64, 65, 83, 99]
(0) (0) (7) (1) (3) (5) (0) (0) (0) (0)



O/p: [4, 7, 64, 65, 83, 99, 124, 333, 545, 723]

Final Sorted O/p
with total Ocmp

Space Compl...
→ $O(n+b)$

Time Complexity.

→ $O(d * (n+b))$
 $\approx O(n * d)$

b = Base of I/p

d = no. of digit in
max element
of given I/p Array

n = no. of elements
in given I/p Array

⇒ Not Inplace

⇒ Stable.

Questions :=

[Que]

If a Radix we use Radix Sort to sort n int. in range $(n^{k/2}, n^k)$, for some $k > 0$, which is independent of n , time taken would be:

- (A) $O(n)$ (B) $O(kn)$ (C) $O(n \log n)$ (D) $O(n^2)$

$$TC: O(n \cdot d)$$

$$(n^{k/2}, n^k)$$

max

$$d = O(\log(n^k)) = O(k \log n)$$

$$\text{If } \therefore 10^3 \rightarrow \log_{10}(10^3) + 1$$

$$d = O(\log n)$$

$$TC = O(n \log n)$$

[Que]

Give the correct matching for the following pairs:

List 1

- A. $O(\log n)$
B. $O(n)$
C. $O(n \log n)$
D. $O(n^2)$

List 2.

- P. Selection
Q. Insertion Sort
R. Binary Search
S. Merge Sort

$$P \rightarrow D$$

$$Q \rightarrow B$$

$$R \rightarrow A$$

$$S \rightarrow C$$

[Que]

If all permutation are equally likely, what is the expected number of inversions in a random chosen permutation of $1, \dots, n$?

- (A) $\frac{n(n-1)}{2}$ (B) $\frac{n(n-1)}{4}$ (C) $\frac{n(n+1)}{4}$ (D) $2n \lceil \log_2 n \rceil$

$$n \rightarrow \text{elements} \Rightarrow {}^nC_2 \text{ pairs} \Rightarrow \frac{n(n-1)}{2}$$

$$\text{Prob}(\text{pair} = \text{inv}) = 1/2$$

$$\text{Expected pairs} = \frac{n(n-1)}{2} \times \frac{1}{2} = \frac{n(n-1)}{4}$$

[Que]

What would be worst case time complexity of the insertion sort algo, if the inputs are restricted to permutation $1 \dots n$ with at most n inversions

(A) $O(n^2)$

(B) $O(n \log n)$

(C) $O(n^{1.5})$

(D) $O(n)$

$$TC = O(n+d)$$

$$d = O(n)$$

$$TC = O(n+n)$$

$$\Rightarrow \underline{O(n)}$$

Note: We use this while solving questions when the I/P sequence / order is not known and only no. of inversions are mentioned

$$TC = \underline{O(n+d)}$$

[Que]

Assume that the algorithm considered here sort the I/P sequence in ascending order. If the I/P is already in ascending order, which of the following is TRUE?

(i) Quick Sort runs in $O(n^2)$ time

(ii) Bubble Sort runs in $O(n^2)$ time $\times$

(iii) Merge Sort runs in $O(n)$ time

(iv) Insertion Sort runs in $O(n)$ time $\checkmark$

(A) I and II only

(B) I and III only

(C) II and IV only

(D) I and IV only $\checkmark$

Ascen... BS $\rightarrow$ Bubble $\rightarrow O(n) \checkmark$
 $\rightarrow O(n^2) \times$

[Que]

Consider following array:

23	32	45	69	72	73	89	97
----	----	----	----	----	----	----	----

Which algo. out of the following options uses the least number of comparisons (among the array element) to sort the above array in ascending order?

(A) Quick Sort using last element as pivot $\rightarrow O(n^2)$

(B) Selection Sort $\rightarrow n^2$ comp.

(C) Merge Sort $\rightarrow O(n \log n)$

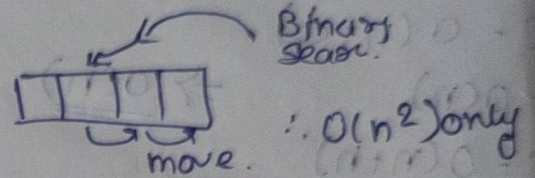
(D) Insertion Sort

$$(n-1) \text{ comp} \\ 2(n)$$

[Que]

The usual $\Theta(n^2)$ implementation of Insertion Sort to sort an array uses linear search to identify the position where an element is to be inserted into the already sorted part of the array. If instead, we use binary search to identify the position, the worst case running time will...

- (A) Remains $\Theta(n^2)$
- (B) Become $\Theta(n(\log n)^2)$
- (C) Become $\Theta(n \log n)$
- (D) Become $\Theta(n)$



$\therefore \Theta(n^2)$ only

...that the algorithm considered here sort the file ...
 ...of the following is true?
 (A) I and II only
 (B) I and III only
 (C) II and IV only
 (D) I and IV only

FF	PS	EF	SF	ES	2N	SE	SE
----	----	----	----	----	----	----	----

...two ...
 ...the following ...
 ...the ...
 ...the ...
 ...the ...